

CS & IT ENGINEERING



Operating System

File System

Lecture - 1

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Multilevel Paging

Topic

Average Memory Access in Multilevel Paging

Topic

Inverted & Hashed Page Table

Topics to be Covered



Topic

Inverted & Hashed Page Table

Topic

File System

Topic

File Allocation Method

→ prerequisite ⇒ Disk structure

→ platter, surface, track,
sector, cylinder, Disk access
time

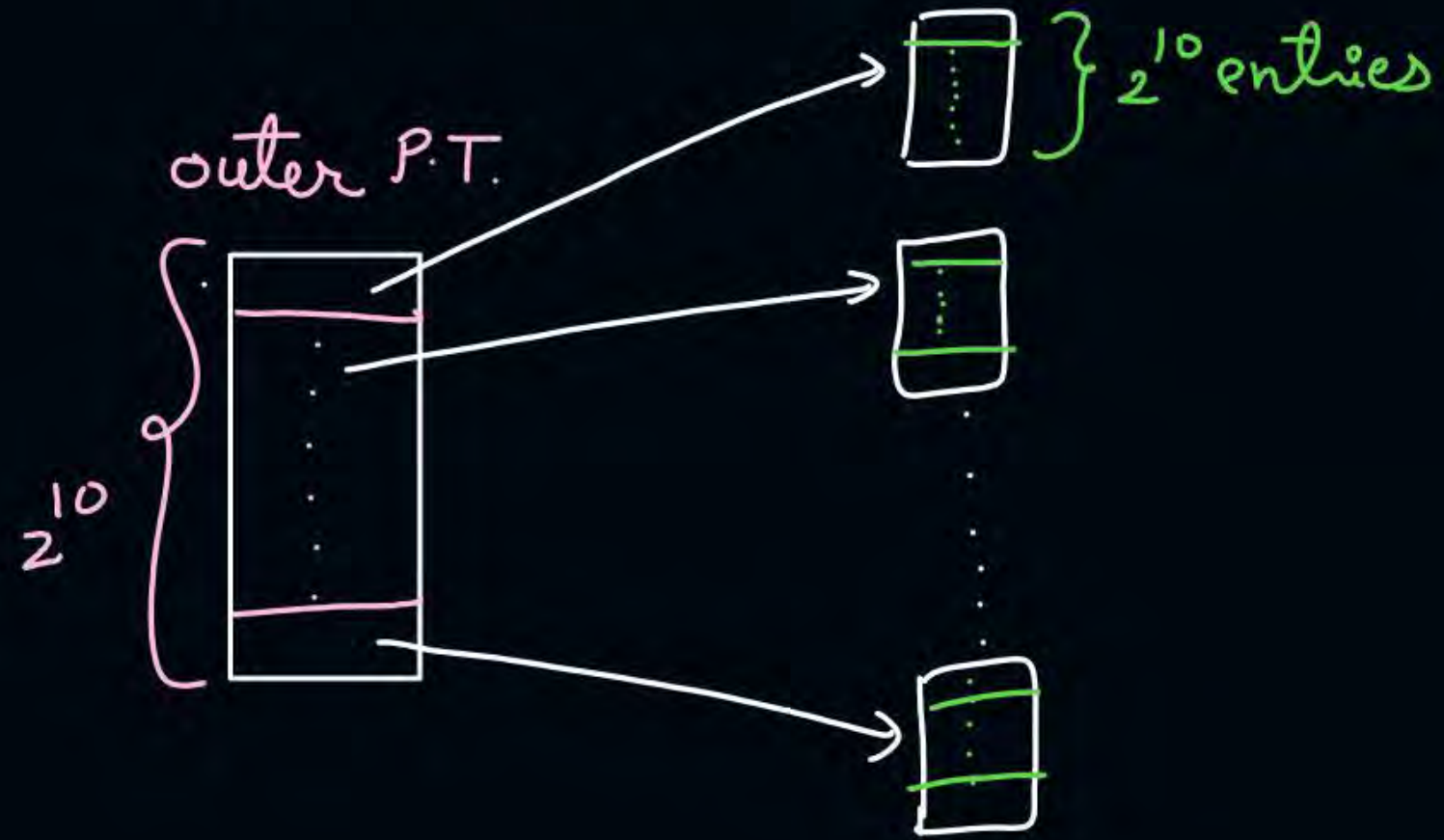
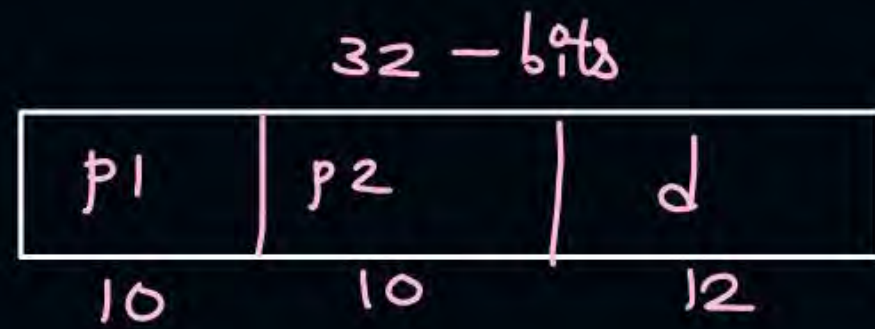


Topic : Question

#Q. Consider a 32-bit system with 4 KB page size and page table entries of size 4 bytes each. Assume $1 \text{ KB} = 2^{10}$ bytes. The OS uses a 2-level page table for memory management, with the page table containing an outer page directory and an inner page table. The OS allocates a page for the outer page directory upon process creation. The OS uses demand paging when allocating memory for the inner page table, i.e. a page of the inner page table is allocated only if it contains at least one valid page table entry.

An active process in this system accesses 2000 unique pages during its execution, and none of the pages are swapped out to disk. After it completes the page accesses, let X denote the minimum and Y denote the maximum number of pages across the two levels of the page table of the process.

The value of $X + Y$ is 1028?



$2^{10} = 1024$
pages
in inner page table

Min. $\Rightarrow 3$

Max $\Rightarrow 1025$

$$x = 3$$

$$y = 1025$$

$$x + y = 1028$$



Topic : Question

#Q. [A processor uses 2-level page tables for virtual to physical address translation. Page tables for both levels are stored in the main memory. Virtual and physical addresses are both 32 bits wide. The memory is byte addressable. For virtual to physical address translation, the 10 most significant bits of the virtual address are used as index into the first level page table while the next 10 bits are used as index into the second level page table. The 12 least significant bits of the virtual address are used as offset within the page.] Assume that the page table entries in both levels of page tables are 4 bytes wide. Further, the processor has a translation look-aside buffer (TLB), with a hit rate of 96%. The TLB caches recently used virtual page numbers and the corresponding physical page numbers. The processor also has a physically addressed cache with a hit rate of 90%. Main memory access time is 10 ns, cache access time is 1 ns, and TLB access time is also 1 ns. Assuming that no page faults occur, the average time taken to access a virtual address is approximately (to the nearest 0.5 ns)?

$$T_{act} (\text{avg content access time with P.A.}) = 0.9 * 1 + 0.1 * (1 + 10) \\ = 2 \text{ ns}$$

$$E.M.A.T. = 0.96 \left(\underbrace{1}_{\substack{\text{TLB} \\ \text{access} \\ \text{to add.} \\ \text{translat}^n}} + \underbrace{2}_{\substack{\text{Tact} \\ \text{with} \\ \text{P.A.}}} \right) + 0.04 * \left(\underbrace{1}_{\substack{\text{TLB} \\ \text{search} \\ \text{miss}}} + \underbrace{2 * 10}_{\substack{\text{2 level} \\ \text{Page table} \\ \text{access for} \\ \text{address translat}^n}} + \underbrace{2}_{\text{Tact}} \right) \\ = 3.8 \text{ ns}$$



Topic : Inverted Page Table

Process

0	
1	
2	
3	
4	
5	
6	
7	

Page Table

0	
1	
2	
3	
4	
5	
6	
7	

↑ page no. ↑ frame no.

Inverted Page Table

0	2
1	5
2	1
3	6

↑ frame no.

Physical Memory

0	Page 2
1	Page 5
2	Page 1
3	Page 6



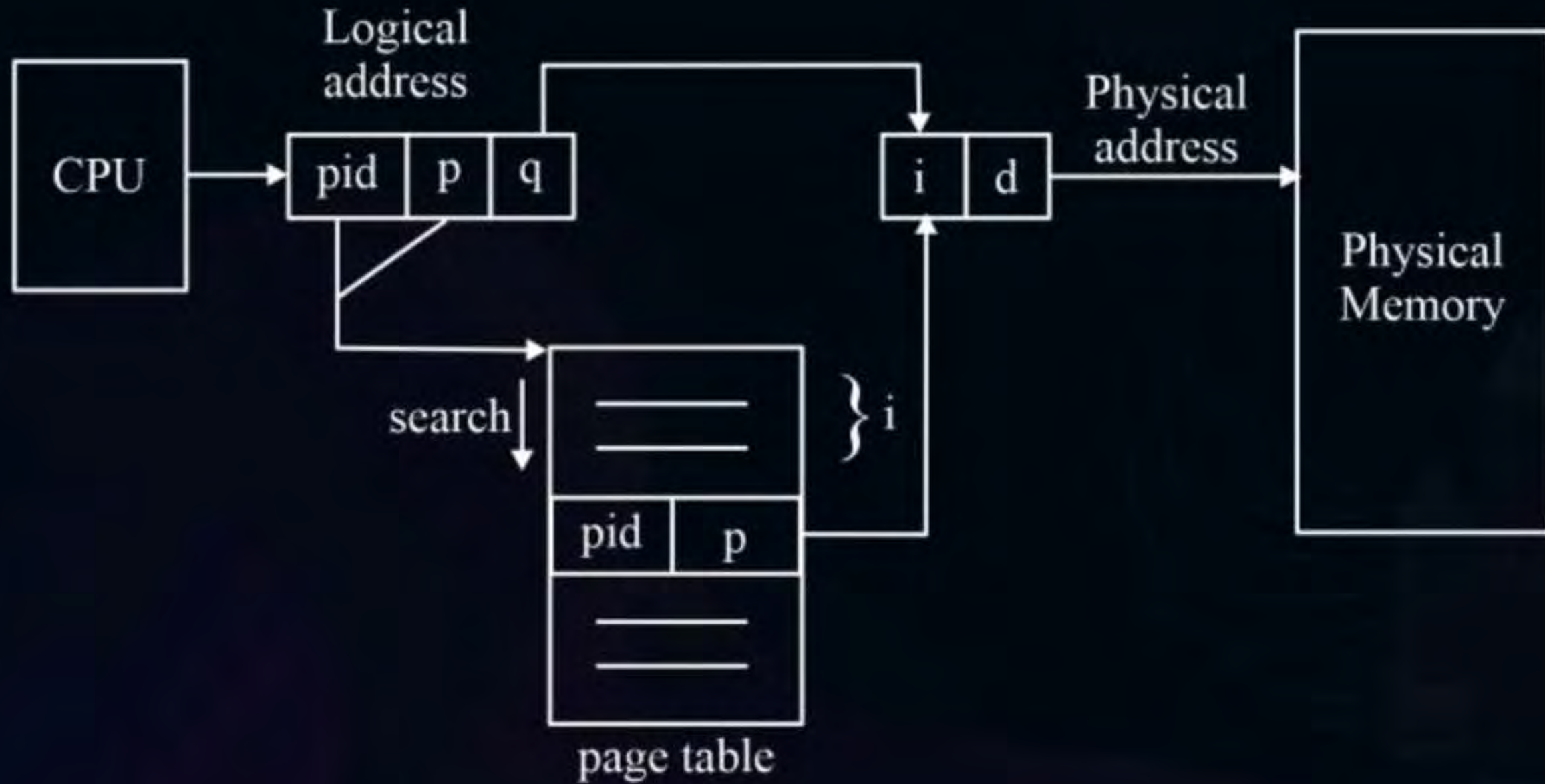
Topic : Inverted Page Table

Each entry in the page table contains the following fields:

1. Page number
2. Process id
3. Control bits
4. Chained pointer



Topic : Inverted Page Table

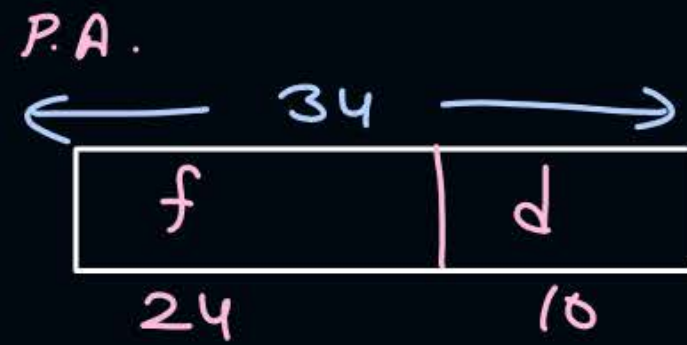
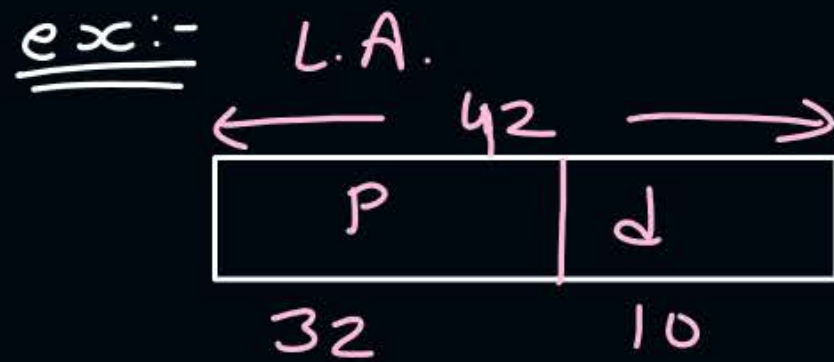




Topic : Inverted Page Table

Advantages & Disadvantages:

1. Reduced memory space
2. Longer lookup time
3. Difficult shared memory implementation



Regular P.T. entry size = 32 bits (24 bits frame no. + extra bits)

Inverted P.T. entry = 40 bits (32 bit page no. + pid + extra bits)

$$\begin{aligned}
 \text{size of P.T.} &= 2^{32} * 32 \text{ bits} \\
 &= 2^{32} * 4 \text{ bytes} \\
 &= 16 \text{ G bytes}
 \end{aligned}$$

$$\begin{aligned}
 \text{size of inverted page table} &= 2^{24} * 40 \text{ bits} \\
 &= 2^{24} * 5 \text{ bytes} \\
 &= 80 \text{ M bytes}
 \end{aligned}$$

Page Sharing :-

P.T. of process P1

5	
	16

P.T. of process P2

10	
	16

mm

frame
no.

16

Page

shared
page

b/w process
P1 and P2

Inverted P.T.

0	Pid	Page no.
	P1	Page 5 or P2 Page 10

16

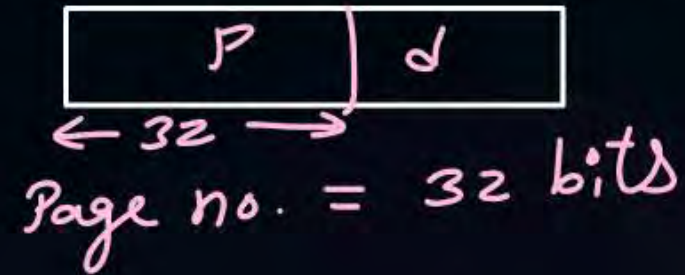
frame
no.



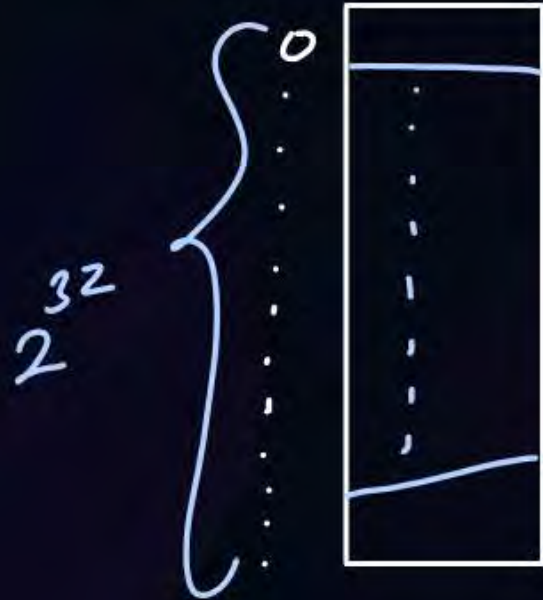
Topic : Hashed Page Table

In this virtual page, the number is hashed into a page table

This Page table mainly contains a chain of elements hashing to the same elements.



no. of P.T. entries = 2^{32}



so many
invalid
entries

16 bit index = Hash function (32 bit page no.)

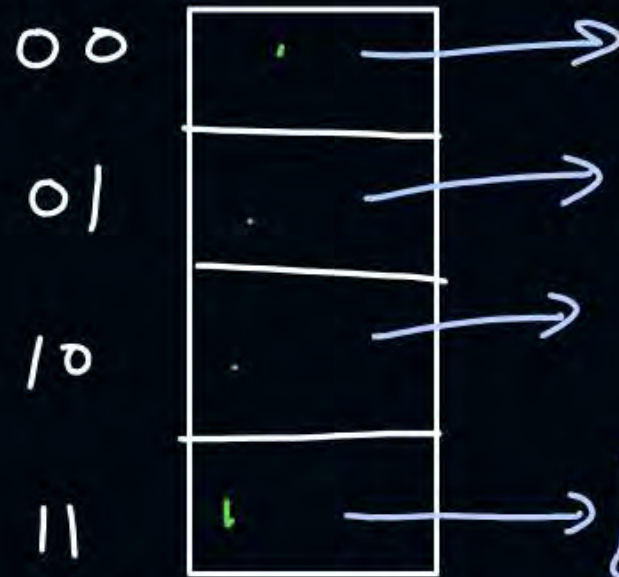
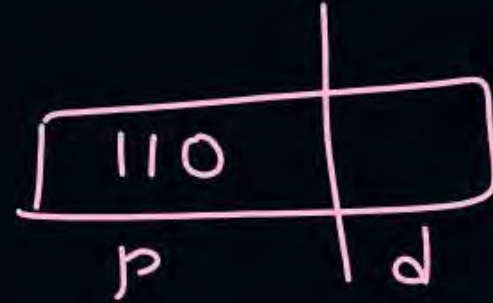
2^{16}



Hashed P.T. will have
lesser no. of P.T.
indexes
for example $\Rightarrow 2^{16}$



if LA

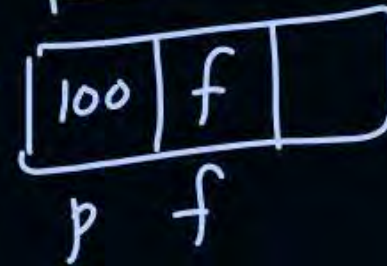
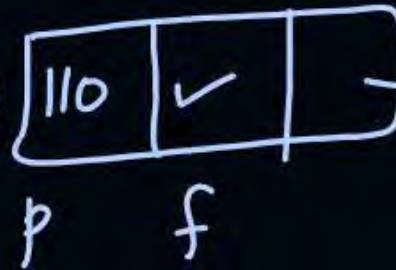


Hashed
p.T.

$H(110) \Rightarrow 2 \text{ bit index no.}$

ex:-

$\Rightarrow 11$





Topic Hashed Page Table

Each element mainly consists of :

1. The virtual page number
2. The value of the mapped page frame.
3. A pointer to the next element in the linked list.

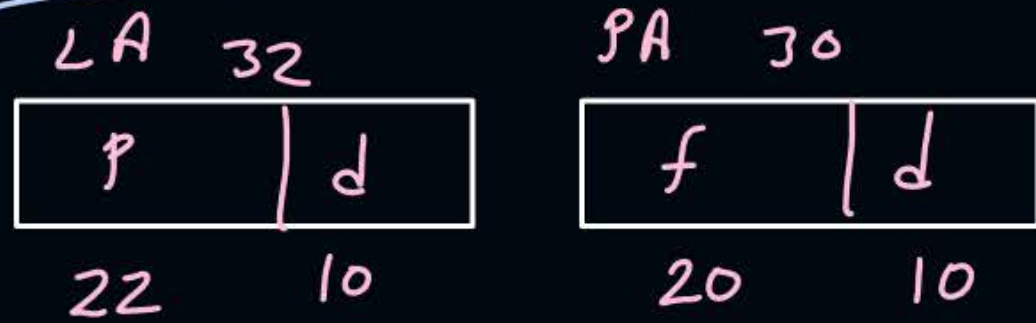
Impact of page size on P.T. size:-

↳ as page size increases, the P.T. size decreases

$$L.A. = 32 \text{ bits}$$

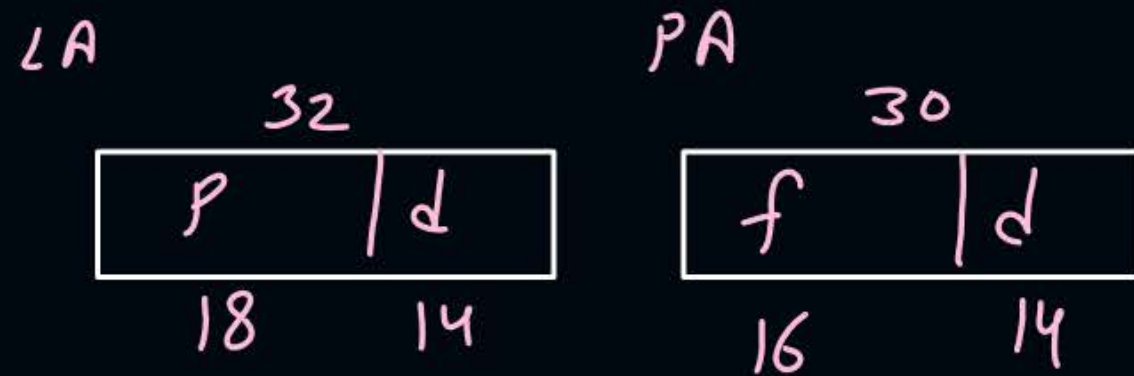
$$P.A. = 30 \text{ bits}$$

Case 1:- Page size = 1KB



$$\begin{aligned} \text{P.T. size} &= 2^{22} * 20 \text{ bits} \\ &= 80 \text{ M bits} \end{aligned}$$

Case 2:- Page size = 16KB



$$\begin{aligned} \text{P.T. size} &= 2^{18} * 16 \text{ bits} \\ &= 4 \text{ M bits} \end{aligned}$$



Topic : File



A file is a named collection of related information that is recorded on secondary storage.



Topic : File Attributes

1. Name
2. Extension *.pptx* *.docx* *.pdf* *.mp3* *.mp4*
3. Size
4. Date
5. Author
6. Created, Modified, Accessed
7. Attributes: Read-only, hidden
8. Default Program
9. Security Details



Topic : File Directory



Collection of files

→ folder

C:\> mkdir rahul ←
↓
creates a folder

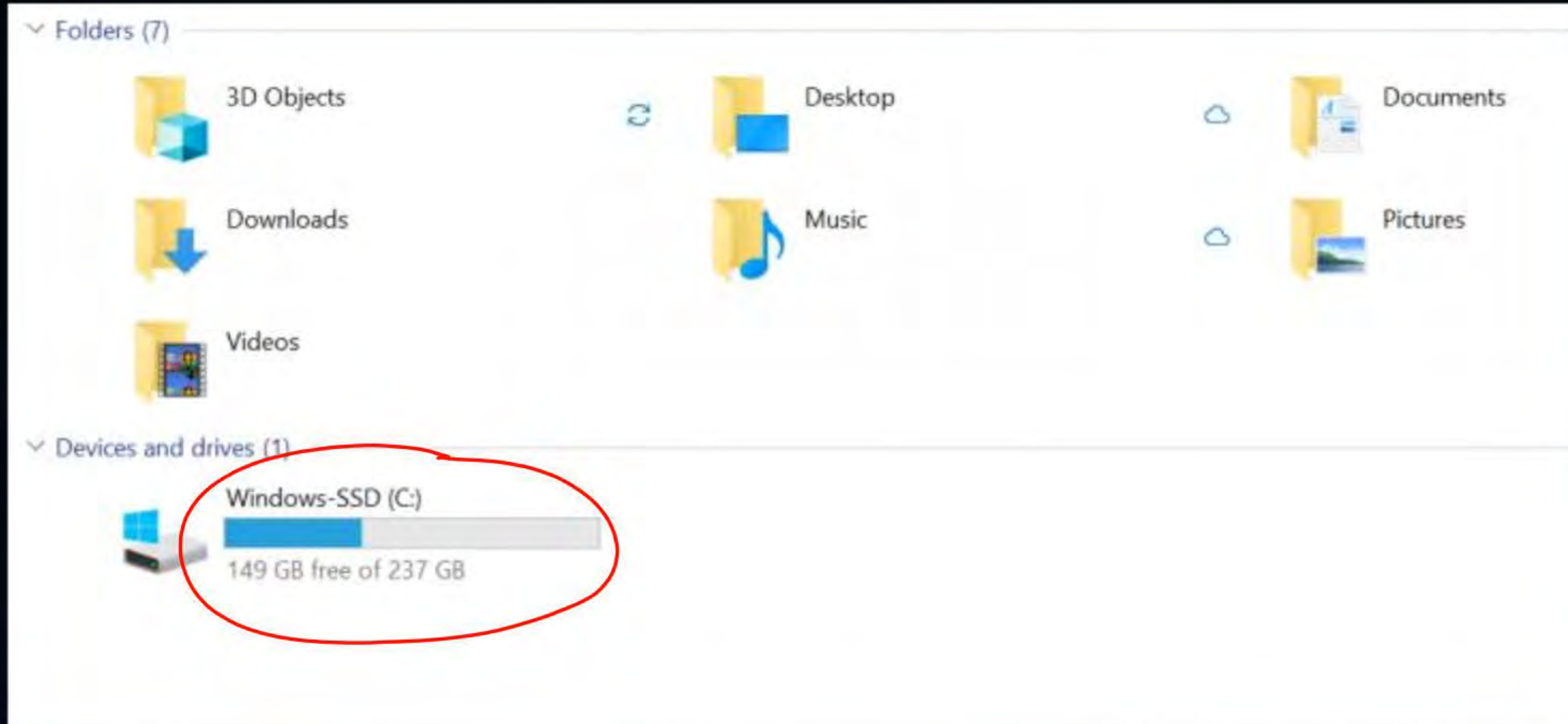


Topic : File System

Module of OS which manages, controls and organizes files and related structures



Topic : File System





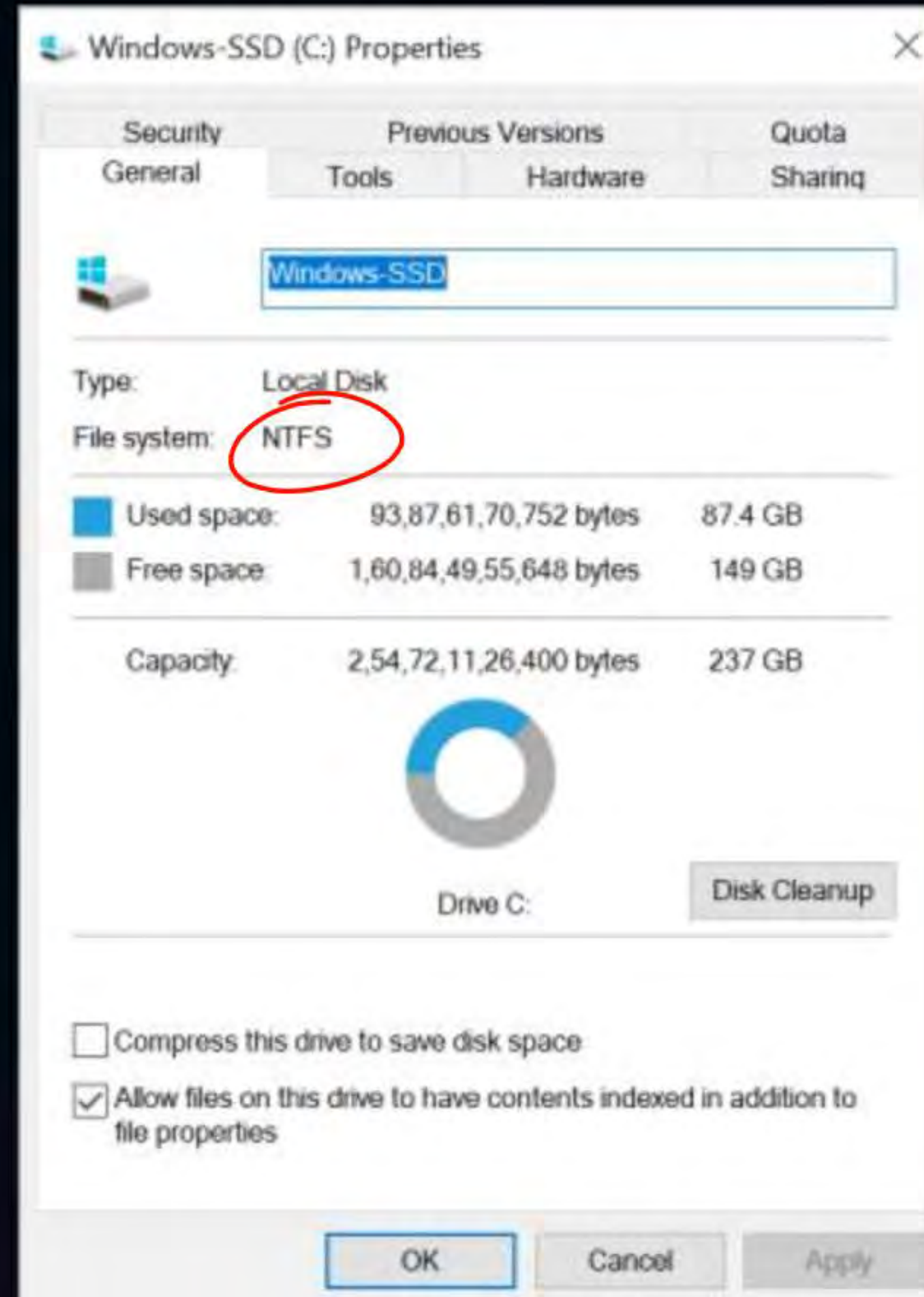
Topic : Types of File Systems

1. FAT32
2. NTFS
3. HFS+
4. Ext2 / Ext3 / Ext4
5. Swap





Topic : File System



Disk formatting

Low-level formatting
or

Physical



making tracks & sectors
on disk

(done by:- manufacturers)

High level formatting
or

Logical



Making disk drives, installing
file system on disk, making disk
blocks, installing OS on disk

Drives on disk (partitions:- C-drive, E-drive)

→ Primary partitions $\Rightarrow$ OS + user files

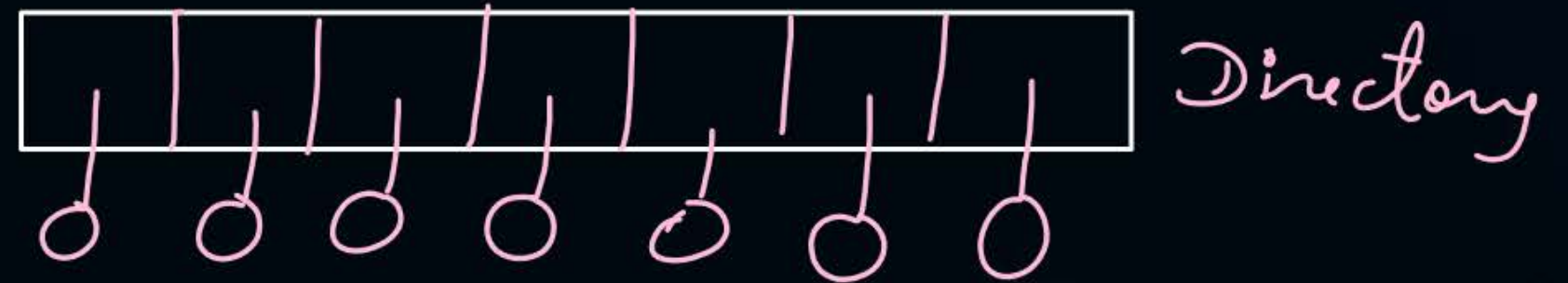
→ Extended partitions $\Rightarrow$ user files



Topic : File Directory Structure

1. Single-Level Directory

```
file_1  
file_2  
file_3  
:  
:  
:  
:  
:
```

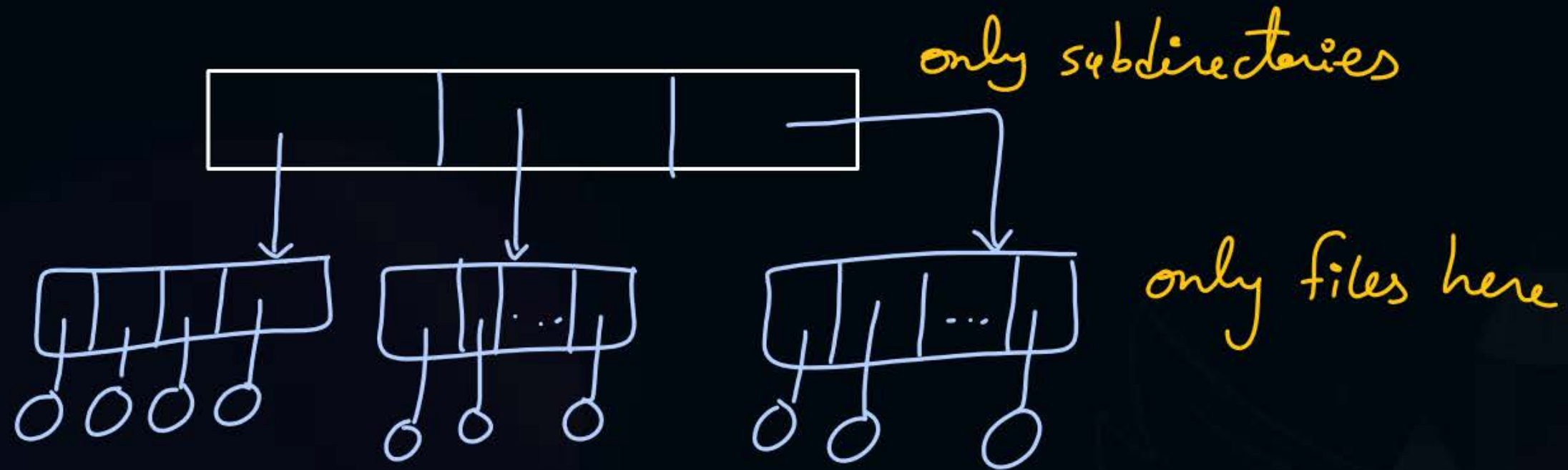


→ All file names must be unique



Topic : File Directory Structure

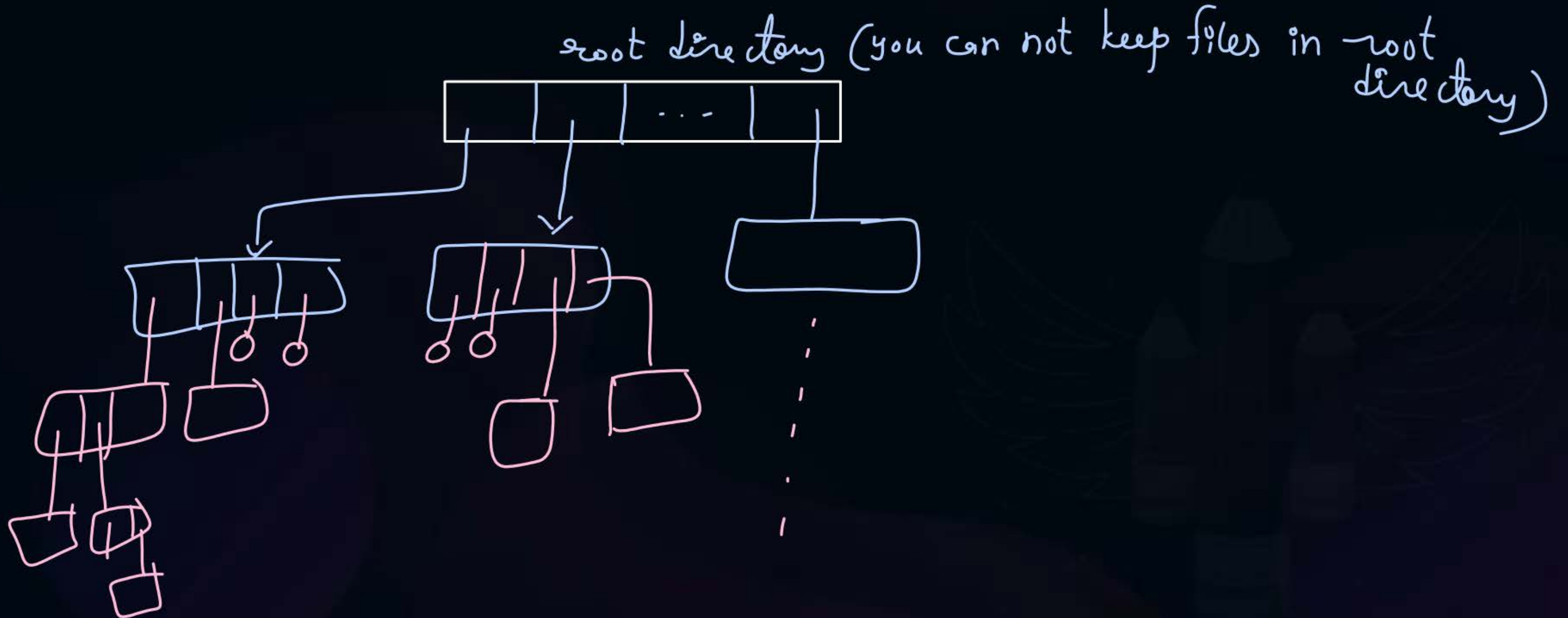
2. Two-Level Directory





Topic : File Directory Structure

3. Tree Structure Directory $\Rightarrow$ latest





Topic : Disk Blocks

In logical formatting $\Rightarrow$ entire disk space is divided in logical disk blocks

1 disk block $\Rightarrow$ 1 sector
or
2 sectors

Plot 1





2 mins Summary

Topic

Inverted & Hashed Page Table

Topic

File System

Topic

File Allocation Method



Happy Learning

THANK - YOU